DriveClique Requirements Specification

Cover Sheet

Field	Value
Project title	DriveClique: A Club and Event Platform for Car Enthusiasts
Document	Capstone Milestone 2: Requirements Specification
Author	Jay Wells
Program	Master of Science in Software Development, Grand Canyon University
Course	SWE-540 Advanced Software Management and Concepts
Instructor	Douglas Mujeye
Document revision	1.0
Date	October 21, 2026

Abstract

DriveClique is a web and mobile platform where car enthusiasts form clubs, schedule group drives, and see who is coming, replacing the mix of social media groups, group chats, and spreadsheets that most car clubs rely on today. This Requirements Specification turns the approved Milestone 1 proposal into testable requirements. It states every functional requirement as an Agile user story with its sequence of actions and acceptance criteria, grouped into ten epics, and states the non-functional requirements as user stories with measurable targets for security, privacy, performance, accessibility, and reliability. It then lists the technologies, frameworks, tools, and hardware the system needs, presents a logical design of how information flows between the clients, the API, the database, and outside services, and gives a sitemap and a wireframe for every screen. The result is a fixed, traceable scope for the development milestones to build against and for the Milestone 4 test plan to verify.

1. Introduction

1.1 Purpose and Background

This specification defines what DriveClique must do and how well it must do it, and gives every requirement an identifier for the Milestone 4 traceability matrix. Car clubs today organize drives in Facebook groups, Discord servers, and group texts, where announcements scroll away, leaders count RSVPs by hand, a full drive has no fair waitlist, and nobody records who showed up. DriveClique gives each club its own space for members, drives, and announcements, with RSVPs, waitlists, check-in, and notifications built in.

1.2 Scope

In scope: accounts and profiles, club discovery and membership, club leadership, drive scheduling with RSVPs and waitlists, check-in, ratings and photos, calendars, notifications (in-app, email, and push), moderation, and analytics. They are delivered as a responsive web app (also installable as a PWA and packaged for Android and iOS) and an Expo mobile app, on one REST API backed by MongoDB.

Deferred to later releases, each keeping its use-case number: drive discussion threads (UC-12), per-drive rating averages (UC-44), two-factor sign-in (UC-47), data export (UC-48), notices when a drive changes (UC-50), email preferences (UC-52), proof of email ownership (UC-54), retention limits (UC-55), session management (UC-56), invite-code replacement (UC-57), an administrator report queue (UC-58), a calendar feed (UC-59), weather forecasts (UC-60), multi-stop routes (UC-61), image storage outside the database (UC-62), and mobile parity (UC-63).

1.3 User Roles

Roles are per club: one account can lead one club and be an ordinary member of another.

Role	Who	Can do, beyond the role above
Visitor	Not signed in	Register, sign in, reset a password
Signed-in user	Any account	Search and create clubs, join or ask to join, manage own profile and settings
Member	Belongs to a club	See its drives, RSVP, check in, rate, report, leave

Role	Who	Can do, beyond the role above
Co-leader	Up to 3 per club	Schedule drives, handle join requests, post announcements, remove members, run check-in, review reports
Leader	The club's owner	Edit or delete any drive, change settings and privacy, appoint co-leaders, transfer or delete the club
Platform administrator	Email listed in `ADMIN_EMAILS`	Open product analytics

1.4 Conventions

Requirements are numbered FR-01 to FR-57 (functional), NFR-01 to NFR-21 (non-functional), and TR-01 to TR-20 (technical), and wireframes W01 to W25. Priority uses MoSCoW: Must, Should, or Could. Status is Built (in the current staging build), Partial (built with a stated gap), or Planned. UC numbers match the code comments and test names.

2. Functional Requirements

DriveClique has 57 functional requirements in ten epics. Each is a user story (As a [role], I want [feature] so that [reason]) followed by its priority, status, and use-case trace, the numbered sequence of actions, and the rules a tester checks.

2.1 Accounts and Access

FR-01 Register an account. As a visitor, I want to create an account so that I can join clubs and RSVP to drives. *(Must · Built · UC-29)* (1) The visitor enters a username, email address, and password, plus an optional name and location, while a meter rates the password Weak, Fair, Good, or Strong. (2) The system checks the address with the email verifier, rejecting disposable or undeliverable domains, and rejects a username or address already in use. (3) The system creates the account, signs the visitor in, and opens the Dashboard. Rules: usernames are 3 to 30 letters, digits, or underscores; passwords are 8 to 100 characters; if the verifier is down, registration proceeds.

FR-02 Sign in and stay signed in. As a registered user, I want to sign in once and stay signed in so that I am not interrupted mid-task. *(Must · Built)* (1) The user enters a username and password. (2) The system issues a 15-minute access token and a 7-day refresh token. (3) When the access token expires, the client renews it and replays the failed request unseen; when the refresh token has expired or been revoked, the client returns to Sign In. Rules: a wrong username and a wrong password get the same message.

FR-03 Sign out. As a signed-in user, I want to sign out so that nobody else on this device can act as me. *(Must · Built)* (1) The user chooses Sign Out from the avatar menu. (2) The system revokes the refresh token, and the client discards both tokens. Rules: a revoked token cannot be renewed.

FR-04 Verify an email address. As a new user, I want to confirm my address from a link so that club email reaches me and nobody else. *(Should · Partial · UC-02, UC-54)* (1) The system emails a verification link. (2) The user opens it, and Verify Email confirms the address. (3) Until then, a Dashboard banner offers to resend it. Rules: club email goes only to verified addresses. Gap: registration currently marks every account verified; turning the check back on is UC-54.

FR-05 Reset a forgotten password. As a user who forgot my password, I want to reset it by email so that I can get back in. *(Must · Built · UC-01)* (1) The user enters an email address on Forgot Password. (2) The system always answers "If that email is registered, a reset link has been sent", and emails a link if the account exists. (3) Within one hour the user sets a new password, and the account's other sessions end. Rules: used or expired links are refused.

FR-06 Change my password. As a signed-in user, I want to change my password so that I can replace one I think is exposed. *(Must · Built · UC-17, UC-29)* (1) Under Security in Profile Settings, the user enters the current password and a new one twice. (2) The system rejects a wrong current password, or a new one matching any of the last five. (3) It saves the new password and ends the other sessions.

FR-07 Change my username. As a signed-in user, I want to change my username so that it reflects how I want to be known. *(Could · Built · UC-21)* (1) The user enters a new username. (2) The system checks its format and that nobody holds it, and saves it. Rules: one change per 60 days; an early attempt says how many days remain.

FR-08 Change my email address. As a signed-in user, I want to move my account to a new address so that I keep receiving club email. *(Should · Built · UC-28)* (1) The user enters the new address. (2) The system rejects one already registered or failing the verifier, and

emails a confirmation link to the new address. (3) Opening the link within 24 hours completes the switch. Rules: the old address keeps working until then.

FR-09 Delete my account. As a user who is leaving, I want to delete my account and what it owns so that none of my data stays behind. *(Must · Built · UC-18, UC-51)* (1) Under Danger Zone, the user confirms with their password. (2) If they lead a club that has other members, the system refuses and names the club. (3) Otherwise it deletes clubs they lead alone, removes them from other clubs, gives their places on upcoming drives to the waitlists, deletes their RSVPs, ratings, sessions, and account, and signs them out.

2.2 Profiles and Personal Settings

FR-10 Edit my profile. As a signed-in user, I want to set my name, photo, bio, and location so that other members recognize me. *(Must · Built)* (1) The user edits first and last name, display-name preference, bio, location, and avatar. (2) The system validates and saves them, and the new name and avatar appear everywhere without a reload. Rules: bio up to 500 characters, name 100, location 200.

FR-11 Keep a garage of my cars. As a car owner, I want to list my cars with photos so that members can see what I drive. *(Should · Built · UC-09)* (1) On Profile, the user adds a car's year, make, model, color, and nickname, uploads photos, and marks a primary car. (2) The system saves the garage, and member lists show each member's primary car. Rules: up to 5 cars with 4 photos each; lists never carry the photos.

FR-12 View another member's profile. As a club member, I want to open another member's profile so that I know who I am driving with. *(Should · Built · UC-22)* (1) The member selects a name in the Members list. (2) A panel shows that person's avatar, name, bio, location, cars, member-since date, the clubs both share, and how many drives they have attended. Rules: if that person has blocked the viewer, the profile answers as if it did not exist.

FR-13 Choose a light or dark theme. As a signed-in user, I want a light or dark theme so that the app is comfortable where I use it. *(Could · Built · UC-49)* (1) In Settings, Appearance, the user picks Dark or Light. (2) The page switches at once, and the choice is saved to the account and applied at every sign-in before the first screen draws.

FR-14 Get started with a checklist. As a new user, I want a checklist of first steps so that I know what to do after signing up. *(Could · Built · UC-26)* (1) The first sign-in shows an onboarding modal. (2) The Dashboard lists Join a club, RSVP to a drive, and Set up your profile, each linking to its screen. (3) Items tick off as they are done; the user can dismiss the list, and it disappears at 3/3.

FR-15 Block another user. As a member, I want to block someone so that they can no longer view my profile. *(Should · Built · UC-32)* (1) On that person's profile panel, the member chooses Block User. (2) The system records the block, and the button becomes Unblock User. Rules: blocking is one-way and never stops a report against the blocker.

2.3 Club Discovery

FR-16 Search for clubs. As a signed-in user, I want to search clubs by name, location, and tag so that I find one that fits my cars. *(Must · Built · UC-15)* (1) On Find Clubs, the user types a search, picks tags, or both. (2) The system returns matching public clubs a page at a time, each with name, location, tags, and member count. (3) A Popular panel shows the most active club, scored as members plus five for each completed drive. Rules: private clubs and clubs the user has blocked never appear; the nine tags are JDM, Muscle, Off-Road, Classic, Luxury, Track, EV, Trucks, and General.

FR-17 Search for clubs near a place. As a signed-in user, I want clubs within a distance of me or of a city so that I join one I can reach. *(Should · Built · UC-46)* (1) The user chooses Use My Location or picks a city, and a radius. (2) The browser asks permission, and the client rounds the position to about 1 km. (3) The system returns clubs inside the radius, nearest first, with their distances. Rules: radius 1 to 250 miles, 25 by default; clubs without a pin are left out, and the panel says so.

FR-18 Find upcoming drives nearby. As a signed-in user, I want upcoming drives near me so that I find an event without browsing club by club. *(Could · Built · UC-46)* (1) With a search point set, Find Clubs lists upcoming drives whose meeting pin is inside the radius, soonest first. Rules: only drives of public clubs or the user's own clubs, minus clubs the user has blocked.

2.4 Club Membership

FR-19 Create a club. As a car enthusiast, I want to create a club so that my group has its own space. *(Must · Built · UC-04)* (1) The user enters a name, a description, and a location (picking a suggestion pins it for distance search), up to five tags, Public or Private, and an optional member limit. (2) The system creates the club with the user as leader and a six-character invite code, and opens it. Rules: a name is 1 to 100 characters and unique; a description is 10 to 1,000.

FR-20 Join a public club. As a signed-in user, I want to join a public club in one click so that I can RSVP right away. *(Must · Built)* (1) The user chooses Join Club. (2) The system adds them, and the page shows the club's drives and member actions. Rules: refused when the

club is full, or when the user is banned from it or has blocked it.

FR-21 Ask to join a private club. As a signed-in user, I want to ask to join a private club so that its leaders can let me in. *(Must · Built · UC-04)* (1) The user chooses Join Club. (2) The system records one pending request, notifies the leader and co-leaders, and shows "Join request sent! Awaiting leader approval."

FR-22 Approve or deny join requests. As a club leader or co-leader, I want to decide join requests so that only people we accept see our private club's drives. *(Must · Built)* (1) The club page lists pending requests to the leader and co-leaders. (2) The reviewer chooses Approve or Deny. (3) The system updates membership and tells the requester in the app and by email. Rules: approval is refused when the club is full; members never see others' requests.

FR-23 Join with an invite code. As a person invited by a friend, I want to enter the club's code so that I can reach a club that search does not show. *(Must · Built)* (1) On Find Clubs or My Clubs, the user chooses Join with Invite Code and enters it. (2) The system previews the club. (3) On confirmation, a public club adds them at once and a private club files a request. Rules: code lookups are limited to 10 a minute per address.

FR-24 Leave a club. As a member, I want to leave a club so that it leaves my schedule and notifications. *(Must · Built · UC-51)* (1) The member chooses Leave Club and confirms a dialog saying they will be taken off its upcoming drives. (2) The system removes them from the members and co-leaders, deletes their RSVPs to drives not yet started, and promotes the next waitlisted member into each place they held. Rules: the leader must transfer ownership or delete the club first; RSVPs to past drives stay.

FR-25 Block a club. As a former member, I want to block a club so that it stops appearing and cannot pull me back in. *(Could · Built · UC-32)* (1) A non-member chooses Block Club. (2) The system hides the club from their searches and refuses their joins, requests, and invite codes for it. (3) A Blocked Clubs panel on Find Clubs lists it with Unblock.

2.5 Club Leadership

FR-26 Edit the club's details. As a club leader, I want to change the name, description, avatar, location, and tags so that the club page stays accurate. *(Must · Built)* (1) Manage Club opens Club Settings at General. (2) The leader edits and saves; only the fields sent change, and the club page and lists update. Rules: switching sections discards unsaved edits; retyping the location without picking a suggestion removes the pin.

FR-27 Make the club public or private. As a club leader, I want to switch the club's privacy so that I control who finds and joins it. *(Must · Built · UC-04)* (1) Under Privacy, the leader picks Public or Private. (2) A private club disappears from search, and its drives become members-only. (3) Only the leader sees the invite code to share. Rules: a non-member gets 403 for a private club's drives.

FR-28 Post announcements. As a club leader or co-leader, I want to post announcements so that members see club news in one place. *(Should · Built · UC-06)* (1) The user writes an optional title and a body and posts it. (2) The system lists it newest first and notifies the members. (3) A leader or co-leader can delete it. Rules: title up to 100 characters, body 1 to 1,000.

FR-29 Appoint co-leaders. As a club leader, I want to make trusted members co-leaders so that running the club does not depend on me alone. *(Should · Built · UC-10)* (1) In the Members list, the leader chooses Promote to Co-Leader or Demote to Member. (2) The system updates the role and notifies the person. Rules: at most three co-leaders; only the leader promotes or demotes.

FR-30 Remove or ban a member. As a club leader or co-leader, I want to remove a member, and optionally ban them, so that I can protect the club. *(Must · Built · UC-32, UC-51)* (1) In the Members list, the user chooses Remove from Club, optionally ticking "Also ban this user from rejoining". (2) The system removes them, deletes their upcoming RSVPs, promotes waitlisted members into their places, and records any ban. (3) The Banned Members panel lists bans, each with Lift Ban. Rules: nobody can remove the leader, and a co-leader cannot remove another co-leader; the removed member is not notified.

FR-31 Transfer ownership. As a club leader, I want to hand the club to another member so that it continues when I step back. *(Must · Built)* (1) Under Ownership, the leader picks a member and confirms. (2) That member becomes leader, and the former leader stays a member. Rules: the new leader must already be a member.

FR-32 Delete a club. As a club leader, I want to delete the club so that a club that has ended leaves nothing behind. *(Must · Built)* (1) Under Danger Zone, the leader chooses Delete Club and enters their email address and a reason. (2) The system deletes the club with its drives, RSVPs, ratings, and reports. Rules: a wrong email address deletes nothing.

FR-33 See how my clubs are doing. As a club leader or co-leader, I want analytics for my clubs so that I see which drives work and who shows up. *(Should · Built · UC-20)* (1) The user opens Settings, Analytics, Clubs. (2) Each club shows members, total and completed

drives, completion rate, most popular drive, most active member, average RSVP rate, checked-in attendance, and average rating. Rules: a fixed number of queries however many clubs; a club with no drives shows "No analytics yet".

2.6 Drive Planning

FR-34 Schedule a drive. As a club leader or co-leader, I want to schedule a drive with a time, a meeting point, and a size limit so that members know where to be and when. *(Must · Built · UC-23)* (1) The user opens Schedule a Drive and enters a name, a date, a start time, and a time zone, which defaults to the browser's. (2) They pick the meeting place, drag the map pin to the exact spot, and add a description, a difficulty (Easy, Medium, or Hard), an attendee limit, and an optional route image. (3) The system stores the local day, time, zone, and exact start instant, and notifies the members in the app and by email. Rules: a start in the past or an unparseable time such as "TBD" is refused; the limit is 1 to 1,000, 100 by default.

FR-35 Schedule a recurring series. As a club leader or co-leader, I want to repeat a drive weekly, every 2 weeks, or monthly so that I set up a regular meet once. *(Should · Built · UC-11)* (1) The user ticks "Make this a recurring drive" and picks a frequency and a number of occurrences. (2) The system creates one linked drive per occurrence at the same local time, even across a daylight-saving change, and notifies the members once. Rules: 2 to 12 occurrences, each editable and cancellable on its own.

FR-36 Edit a drive or mark it complete. As a club leader, I want to correct a drive and mark it complete so that details stay accurate and members can rate it. *(Must · Built)* (1) The leader chooses Edit Drive, changes any field, and saves. (2) After the drive, Mark Complete closes check-in and opens ratings and the photo gallery. Rules: only the leader can edit or complete a drive.

FR-37 Cancel a drive or the rest of a series. As a club leader, I want to cancel a drive with a reason so that nobody shows up for a drive that is not happening. *(Must · Built)* (1) The leader chooses Cancel Drive, enters a reason, and may cancel every later drive in its series. (2) The system marks the drives cancelled and notifies the members in the app and by email. (3) The drive leaves the club's drive lists, and RSVP histories mark it Cancelled. Rules: a reason is required; a co-leader may cancel only drives they created; only the leader cancels a series.

FR-38 Delete a drive. As a club leader, I want to delete a drive created by mistake so that it does not clutter the club's history. *(Should · Partial)* (1) The leader chooses Delete Drive and confirms. (2) The system deletes the drive and its RSVPs. Gap: the drive's ratings are

not yet deleted with it.

FR-39 View a drive's details. As a club member, I want to open a drive and see everything about it so that I can decide whether to go. *(Must · Built · UC-23, UC-53)* (1) The member selects a drive on the club page, the Dashboard, or the Calendar. (2) The system loads the drive with its route image and photos, the RSVP counts, the member's own answer, and the ratings. (3) The panel shows the time in the drive's zone and in the member's own when it differs, as in "10:00 AM MST (1:00 PM your time)", plus the map, the details, and the actions the member's role allows. Rules: drive lists never carry images; a non-member of a private club gets 403.

2.7 Attendance

FR-40 RSVP to a drive. As a club member, I want to answer Going, Maybe, or Not Going so that the organizer knows who to expect. *(Must · Built · UC-03)* (1) The member chooses an answer in the drive panel. (2) The system records it, updates the counts, and notifies the drive's organizer. Rules: members of the club only; one RSVP per member and drive, replaced when changed.

FR-41 Wait for a place on a full drive. As a club member, I want to join a waitlist when a drive is full and be moved up automatically so that I get a place fairly. *(Must · Built · UC-03, UC-51)* (1) Choosing Going on a full drive puts the member on the waitlist and shows their position, as in "You joined the waitlist at position #2". (2) When a Going member changes their answer, leaves the club, is removed, or deletes their account, the first person in line becomes Going. (3) The promoted member is notified in the app and by email. Rules: position comes from join order when read, so nothing is renumbered; choosing Going again while waitlisted adds no duplicate.

FR-42 See who is coming. As a club leader or co-leader, I want the attendee list for a drive so that I can plan for the people coming. *(Should · Built · UC-10)* (1) The user opens the Attendee List in the drive panel. (2) The system lists usernames grouped by answer. Rules: ordinary members see only the counts; the list carries no email addresses.

FR-43 Check in at a drive. As a club leader, I want attendees to confirm they are present so that the club records who actually came. *(Should · Built · UC-08)* (1) The leader or a co-leader chooses Send Check-In Notification. (2) Every member who answered Going receives it and opens the Drive Check-In page. (3) Each marks Present or Not Present, and the leader sees the Present, Not Present, and Pending counts and can resend. Rules: only Going members can check in; check-in closes when the drive is marked complete.

FR-44 Rate a drive. As a member who went on a drive, I want to rate it so that leaders learn what made it good. *(Should · Built · UC-25)* (1) After the drive is complete, the member picks one to five stars and adds an optional comment. (2) The system saves the rating and shows the drive's average and count. Rules: only Going members can rate; one rating per member, which they can change; a comment holds up to 200 characters.

FR-45 Share photos from a drive. As a club leader or co-leader, I want to add the best photos to a completed drive so that members can look back on it. *(Could · Built · UC-05)* (1) The user uploads photos in the drive panel. (2) The system adds them and notifies the attendees, and a leader or co-leader can remove one. Rules: up to 12 photos, only after the drive is complete.

FR-46 Review my RSVP history. As a member, I want a history of the drives I answered so that I can see how active I have been. *(Could · Built · UC-19)* (1) The member opens Settings, Analytics, Personal. (2) The page shows total RSVPs, drives attended, upcoming drives, and a favorite club, then each drive marked upcoming, past, or cancelled, including drives of clubs the member has since left.

2.8 Schedule and Calendar

FR-47 See my upcoming drives at a glance. As a member of several clubs, I want one list of my upcoming drives so that I do not have to visit each club. *(Must · Built)* (1) The Dashboard shows up to ten upcoming drives from all the user's clubs, soonest first, each with its club and going count. (2) The Home item in the sidebar shows a badge with the total. (3) Selecting a drive opens its panel. Rules: one request each, however many clubs; a failed request says "Couldn't load upcoming drives", never "No upcoming drives".

FR-48 Browse a month calendar. As a member, I want a month calendar of my clubs' drives so that I can plan around them. *(Should · Built · UC-24)* (1) Calendar shows the month with drive days outlined. (2) The user moves between months, a day previews its drives, and a list beside the grid shows the month's drives. (3) Selecting a drive opens its panel. Rules: each drive appears on its own local day, wherever the viewer is.

FR-49 Add drives to my own calendar. As a member, I want to export drives to my phone's calendar so that I get its reminders. *(Could · Built · UC-33)* (1) Add to Calendar in a drive panel returns an .ics file for that drive. (2) Personal analytics exports one .ics file with every upcoming drive the member answered Going or Maybe. Rules: start times are written in UTC, so any calendar shows the right time; members only.

2.9 Notifications

FR-50 Receive live notifications. As a member, I want a notification the moment something happens in my clubs so that I miss nothing. *(Must · Built · UC-30)* (1) While signed in, the client keeps a notification stream open. (2) A new drive, a cancellation, a waitlist promotion, or a join decision adds to the bell's unread count. (3) The user reads the history, filters to Unread, opens an item to go to what it concerns, and marks one or all as read. Rules: delivered within 5 seconds to an open browser; the 16 types are listed in Section 5.5.

FR-51 Choose which notifications I get. As a member, I want to turn off kinds of notification so that the ones I keep stay useful. *(Should · Built · UC-30, UC-52)* (1) Under Notifications in Profile Settings, the user sees one switch per type. (2) The system saves the switches and skips the in-app notification and push for any type switched off. Rules: a type added later starts switched on. Gap: the switches do not yet govern email (UC-52).

FR-52 Receive important news by email. As a member, I want email for the events that matter so that I hear about them with the app closed. *(Should · Built)* (1) New and cancelled drives, waitlist promotions, join decisions, drive reminders, and new reports also send an email. (2) Account email (password reset, verification, email change) is always sent. Rules: club email goes only to verified addresses; with no SMTP settings, email is skipped and everything else works.

FR-53 Get a reminder before a drive. As a member going to a drive, I want a reminder about a day ahead so that I do not forget it. *(Should · Built · UC-04, UC-51)* (1) At the top of every hour, the scheduler finds drives that start within 24 hours and are not cancelled. (2) Each member who answered Going or Maybe and has not been reminded gets a notification and an email. Rules: each RSVP is reminded once; former members are not reminded.

FR-54 Receive push notifications. As a mobile app user, I want push notifications so that I hear about my clubs without opening the app. *(Should · Built · UC-34)* (1) After sign-in, the mobile app asks permission and registers the device's push token. (2) Every notification for the user is also pushed through the Expo Push API to each registered device. (3) Signing out removes the device's token. Rules: up to 10 devices per user, with no duplicates.

2.10 Safety, Moderation, and Insight

FR-55 Report a club, a drive, or a member. As a member, I want to report something harmful so that someone responsible can act. *(Must · Built · UC-14, UC-42)* (1) The member chooses Report on a club, a drive, or a member in the Members list. (2) They pick Harassment, Spam, Dangerous, or Other, and add optional details. (3) A report on a drive, or on a member reported from a club, joins that club's review queue, and its leader and co-leaders are notified. Rules: one report per reporter and target, reopened if reported again after it is closed; details up to 500 characters. Gap: reports on a club itself reach no reviewer yet (UC-58).

FR-56 Review reports for my club. As a club leader or co-leader, I want a queue of reports about my club's drives and members so that I can deal with each one. *(Should · Built · UC-42)* (1) The reviewer opens Club Settings, Reports, from Manage Club, Review Reports, or the notification. (2) Open reports show the reason, details, reporter, and the target's current state, such as "Cancelled", 20 per page. (3) Resolve or Dismiss moves a report to Closed, where it can be reopened. Rules: a reviewer never sees or closes a report about themselves; no email addresses appear.

FR-57 See how the product is used. As a platform administrator, I want activity totals across all users so that I can see which features people use. *(Could · Built · UC-41)* (1) Each route change records a page view, and key actions (club created or joined, drive scheduled, RSVP, rating, report) record an event. (2) Product Analytics shows totals by event type and the most viewed pages. Rules: only accounts listed in `ADMIN_EMAILS`; anyone else sees "You don't have access to this page".

3. Non-Functional Requirements

DriveClique has 21 non-functional requirements. Each is a user story with a measurable target and the steps that verify it; where an automated test already checks the target, it is named. Sections 3.7 and 3.8 give the security matrix and the security risks still open.

3.1 Security

NFR-01 Short-lived, revocable sessions. As a user, I want a stolen sign-in to stop working quickly so that a copied token is of little use. Target: access tokens expire after 15 minutes and refresh tokens after 7 days; refresh tokens are stored only as SHA-256 hashes and expire through a TTL index; signing out revokes the session, and a password change or reset revokes all of them; passwords are bcrypt hashes (cost 10) and never returned.

Verify: (1) decode an access token: it expires 15 minutes after issue; (2) call a protected route with an expired token: 401, then the client refreshes and replays; (3) present a refresh token after sign-out: 401.

NFR-02 Permissions enforced by the server. As a club leader, I want every permission checked by the server so that nobody can act as a leader by calling the API directly. Target: every write to a club, drive, RSVP, photo, or report checks the caller's role in that club and answers 403 when the role is not allowed (Section 3.7); clients never request members-only data for non-members. Verify: (1) send each leader-only request as an ordinary member: 403, nothing changes; (2) request a private club's drives as a non-member: 403; (3) open a private club as a non-member: no members-only request appears in the network log.

NFR-03 Limits on request volume. As the platform operator, I want each client address rate-limited so that password guessing, mass sign-ups, and scraping are slowed down. Target, per address in production: 120 API requests a minute; 10 invite-code lookups a minute; 10 sign-ins in 15 minutes; 5 registrations an hour; 3 password-reset, verification, and email-change requests an hour each; one trusted proxy hop on Railway so each client is counted by its own address. Verify: (1) the eleventh sign-in within 15 minutes answers 429; (2) a second address is counted separately (`security-hardening.spec.ts`).

NFR-04 Validated input everywhere. As the platform operator, I want every request checked before it reaches the database so that hostile input cannot corrupt data or inject queries. Target: every route validates its body, query, and path, answering 400 with a readable message; ids must be 24 hexadecimal characters; a query parameter sent twice or as an object is refused, so it can never become a MongoDB operator; Section 2's length limits are enforced; email addresses are capped at 254 characters. Verify: (1) `/api/clubs/browse?query[$ne]=x` answers 400; (2) a 101-character drive name answers 400; (3) a malformed id answers 400, not 500.

NFR-05 Hardened transport and browser. As a user, I want the app protected from interception and malicious pages so that my session cannot be stolen. Target: HTTPS in production; Helmet's headers on the API; cross-origin calls allowed only from the web app and the Capacitor apps; the web server sends a Content-Security-Policy with scripts from its own origin only, `X-Frame-Options: DENY` , `nosniff` , a strict referrer policy, and a Permissions-Policy that allows geolocation on its own origin only. Verify: (1) read the production response headers; (2) a cross-origin call from another site fails.

NFR-06 No known vulnerable dependencies. As the developer, I want vulnerable dependencies caught before release so that a published flaw cannot ship. Target: no high or critical `npm audit` advisory in the API's dependencies or the web app's production dependencies, and no unresolved high-severity code-scanning alert. Verify: (1) CI runs `npm audit --audit-level=high` on every push and blocks deployment on failure; (2) review code-scanning alerts before each merge to `main`.

3.2 Privacy

NFR-07 Only the data each viewer needs. As a member, I want my email address and my club's private details kept from others so that joining a club does not expose me. Target: no club response carries anyone's email address; only the leader receives the invite code; no club read carries the ban list; join requests go to the leader and co-leaders, and anyone else sees only their own; every club response names the fields it returns. Verify: (1) read one private club as a member, a non-member, a co-leader, and the leader: only the leader's response holds the invite code, and none holds an email address (`club-data-privacy.spec.ts`).

NFR-08 Location shared only on request. As a user, I want my location used only when I ask so that the app never tracks me. Target: the browser's location is read only on Use My Location, rounded to two decimal places (about 1 km) before it leaves the device, and never stored. Verify: (1) inspect the search request: coordinates have two decimals; (2) no user document holds coordinates.

NFR-09 Nothing left behind. As a user who leaves, I want my data actually deleted so that none of it outlives my account. Target: deleting an account removes, in one request, the account, its RSVPs, ratings, and sessions, the clubs it led alone, and its place in every other club; deleting a club removes its drives, RSVPs, ratings, and reports. Verify: (1) delete a seeded account and query every collection for its id: no rows remain.

3.3 Performance and Resource Use

NFR-10 Light first load. As a member on a phone connection, I want the app to open quickly so that I can check a drive on the spot. Target: the JavaScript needed for first load stays under 250 KB gzipped, with each page's code loaded only when it opens (measured 181 KB on the 2026-10-20 build); text assets are gzip-compressed and fingerprinted assets cached for a year. Verify: (1) build the app and gzip the files the first page requests; (2) open the Dashboard: other pages' code is not requested.

NFR-11 Lean responses. As a member, I want lists without images I am not viewing so that pages stay fast and use little data. Target: drive and member lists never carry route images, galleries, or car photos; one drive's images load when its panel opens; each stored image is at most 300,000 characters. Verify: (1) a drive with a full gallery has no `image` or `photos` in its club's list (`list-payload-size.spec.ts`).

NFR-12 A fixed number of requests and queries. As a member of many clubs, I want pages to cost the same however many clubs I join so that the app does not slow down as I use it more. Target: the Dashboard and the sidebar make one request each for upcoming drives; club analytics runs a fixed number of batch queries; the trending club takes one aggregation. Verify: (1) as a member of five clubs, the Dashboard makes two requests to the drives API (`dashboard-requests.spec.ts`).

NFR-13 Responsive API. As a member, I want every screen to answer promptly so that using the app never feels like waiting. Target: with 50 concurrent users on the production tier, 95% of reads complete within 500 ms and 95% of writes within 1,000 ms; every frequent query uses an index. Verify: (1) seed a club with 1,000 drives and 200 members; (2) run a 5-minute, 50-user load script on staging and read the 95th percentiles (scheduled for Milestone 4); (3) run `explain()` on each frequent query: no collection scans.

NFR-14 A bounded database. As the platform operator, I want the database to grow only with real use so that the project fits its hosting budget. Target: every list inside a document is capped (3 co-leaders, 5 tags, 5 cars, 4 photos per car, 12 photos per drive, 12 drives per series, 10 push tokens); expired refresh tokens are removed by a TTL index; no collection gains one row per message without a cap or expiry; staging fits a free Atlas M0 cluster (512 MB). Verify: (1) try to exceed each cap through the API: the extra item is refused or dropped.

3.4 Reliability and Availability

NFR-15 Degrades instead of failing. As a member, I want the app to keep working when an outside service fails so that one outage does not stop me from RSVPing. Target: with no SMTP settings, email is skipped; if the email verifier is down, registration proceeds; if geocoding fails, a typed location saves without a pin; a dropped notification stream reconnects; a screen whose data fails to load says so. Verify: (1) run with no SMTP and the verifier on a closed port, then register, schedule a drive, and RSVP (as CI does); (2) fail the upcoming-drives request: the Dashboard says "Couldn't load upcoming drives".

NFR-16 Consistent data under concurrent use. As a member, I want RSVPs and waitlists to stay correct when many people act at once so that nobody is counted twice or skipped. Target: unique indexes allow one RSVP and one rating per member and drive and one report per reporter and target, and keep usernames, email addresses, and club names unique; waitlist position is derived from join order when read. Verify: (1) three members choose Going on a full drive in quick succession: positions 1, 2, and 3; (2) the same RSVP sent twice in parallel leaves one row.

NFR-17 Only tested code is deployed. As a user, I want each release to have passed its tests so that an update does not break the app. Target: Railway deploys a commit only after CI passes lint, a production build, the dependency audit, and the end-to-end suite on a fresh MongoDB 7; the API answers a health check on `/`; each environment has its own database. Verify: (1) push a commit that fails one test: it does not go live.

3.5 Usability and Accessibility

NFR-18 Accessible to everyone. As a member who uses a keyboard or a screen reader, I want every screen to work without a mouse so that I can use the app independently. Target: WCAG 2.1 level AA; no critical or serious axe-core violation; text contrast of at least 4.5:1 in both themes; every action reachable by keyboard, a skip-to-content link, focus kept inside open dialogs, and Escape closing only the top overlay; `jsx-a11y` lint passes. Verify: (1) CI runs axe-core (`accessibility.spec.ts`) and lint on every push; (2) tab through every screen in the sitemap; (3) read the main flows with a screen reader (scheduled for Milestone 4).

NFR-19 Usable on any screen and in any time zone. As a member, I want the app to fit my phone and show times I can trust so that I arrive at the right time. Target: every screen works from 360 px wide to desktop, with side panels collapsing into drawers; every drive time shows the drive's own zone and, when different, the viewer's; daylight-saving gaps and overlaps resolve to a real instant. Verify: (1) run the drive flows at 360 px and 1440 px; (2) view a Pacific-time drive from a browser set to New York time (`drive-time-zones.spec.ts`); (3) run `driveTime.test.js` across a daylight-saving change.

3.6 Portability and Maintainability

NFR-20 Runs where members are. As a member, I want DriveClique on my laptop and my phone so that I can plan at home and check in at the drive. Target: the web app runs in current Chrome, Edge, Firefox, and Safari, installs as a PWA, and builds as Android and iOS apps through Capacitor; the Expo app runs on Android and iOS; the API runs on Node.js

20.19 or newer; the whole stack starts with Docker Compose. Verify: (1) CI runs the suite in Chromium; (2) before each milestone release, run the main flows in Firefox and Safari and on one Android and one iOS device.

NFR-21 Easy to change safely. As the developer, I want the code tested and consistent so that I can add features without breaking old ones. Target: every functional requirement has an end-to-end test and every fixed bug a regression test; errors reach clients in one shape, `{ success: false, message }`, from one handler. Verify: (1) the Milestone 4 traceability matrix maps each FR to a passing test (today about 460 end-to-end tests in 58 files, plus 18 backend unit tests); (2) sample error responses all share that shape.

3.7 Security Matrix

Rights accumulate from left to right among the club roles: a co-leader can do everything a member can, and the leader everything a co-leader can. The server enforces every cell (NFR-02).

Action	Visitor	Signed-in user	Member	Co-leader	Leader	Platform admin
Register, sign in, reset a password	✓					
Edit own profile, garage, theme, and notification settings; delete own account		✓	✓	✓	✓	✓
Search clubs, open a club page, create a club		✓	✓	✓	✓	✓
Join a public club, ask to join a private one, use an invite code		✓				
Report a club		✓	✓	✓	✓	
View a private club's drives and RSVP counts			✓	✓	✓	
RSVP, check in, rate a drive, export to calendar			✓	✓	✓	
Report a drive or a member			✓	✓	✓	
Leave the club			✓	✓	Transfer first	
Schedule a drive or a series				✓	✓	
Cancel a drive				Own drives	✓	

Action	Visitor	Signed-in user	Member	Co-leader	Leader	Platform admin
Send check-in notifications, add or remove photos				✓	✓	
View the attendee list				✓	✓	
Post or delete announcements				✓	✓	
Approve or deny join requests				✓	✓	
Remove or ban members, lift bans				Members only	✓	
Review the club's reports				✓	✓	
View club analytics				✓	✓	
Edit, complete, or delete a drive; cancel the rest of a series					✓	
See the invite code, edit details and privacy					✓	
Promote or demote co-leaders, transfer ownership, delete the club					✓	
Open product analytics						✓

3.8 Open Security Risks

These are known, accepted for now, and tracked to a fix.

Risk	Effect	Planned fix
Email ownership is not proven at registration	Someone can register with another person's address, who then receives club email	UC-54
Refresh tokens are not rotated on use	A copied refresh token works for its full 7 days	UC-56
Email templates insert user text without escaping	A drive or club name could inject HTML into an email	Escape every template
The API accepts JSON bodies up to 50 MB, because images travel as base64	Large requests cost memory and bandwidth	UC-62

Risk	Effect	Planned fix
The notification stream authenticates with a token in its URL, since `EventSource` cannot send headers	The token could appear in logs	Mitigated: the request log redacts it

4. Technical Requirements

DriveClique is three JavaScript applications on one API: a React web app, an Expo mobile app, and a Node.js and Express API over MongoDB, hosted on Railway and MongoDB Atlas. Versions are those in the project's package manifests on 2026-10-21.

4.1 Requirements and Technologies

ID	Requirement	Technology and version
TR-01	A REST API over HTTPS that exchanges JSON, every route under `/api`	Node.js 20, Express 5.2
TR-02	Stateless, revocable authentication	jsonwebtoken 9 (15-minute tokens), refresh tokens hashed in MongoDB (7 days), bcryptjs 3
TR-03	A document store with validation, expiry, and geospatial search	MongoDB 7, Mongoose 9.6, TTL and `2dsphere` indexes
TR-04	Server-to-browser notifications without polling	Server-Sent Events from a Node `EventEmitter`
TR-05	Scheduled background work	node-cron 4.2, hourly drive reminders
TR-06	Outgoing email that can be switched off	Nodemailer 10 over SMTP
TR-07	Push notifications to phones	Expo Push API, tokens registered by the mobile app

ID	Requirement	Technology and version
TR-08	Maps, place search, and distance search	Leaflet 1.9 and react-leaflet 5 on OpenStreetMap tiles, Nominatim geocoding, MongoDB `$geoNear`, haversine distance
TR-09	Correct times across zones and daylight saving	IANA zones with the built-in `Intl` API (`utils/driveTime.js`, `lib/dateUtils.js`)
TR-10	Standard calendar files	iCalendar (RFC 5545) `.ics`, UTC start times
TR-11	A single-page web app that loads each page's code on demand	React 19.2, React Router 7.15, Vite 8, Axios 1.19
TR-12	An installable web app	vite-plugin-pwa 1.3 (web manifest and service worker)
TR-13	Android and iOS builds of the web app	Capacitor 8.4
TR-14	A native mobile client	Expo SDK 56, React Native 0.85, Expo Router, expo-notifications, expo-secure-store, NativeWind 4.2
TR-15	Two colour themes without rewriting components	Tailwind CSS 3.4 mapped to CSS variables, Radix UI, lucide icons
TR-16	Security middleware	helmet 8, cors 2.8, express-rate-limit 8.5
TR-17	Automated end-to-end and accessibility tests	Playwright 1.60 with axe-core 4.11, Node's test runner for units
TR-18	A gate that blocks untested code from deploying	GitHub Actions: ESLint 10 with `jsx-a11y`, production build, `npm audit`, Playwright on MongoDB 7
TR-19	Reproducible builds and managed hosting	Docker (`node:20-alpine`, `nginx:alpine`), Docker Compose, Railway, MongoDB Atlas
TR-20	Logs without secrets, configuration outside the code	winston 3.19 and morgan 1.11 with tokens redacted, `.env` variables

4.2 Third-Party Services

Service	Used for	When it fails
MongoDB Atlas	Database, one per environment	The API cannot serve requests: the only hard dependency
Railway	Hosting the API and the web app	The app is offline
OpenStreetMap (Nominatim and tiles)	Place search, coordinates, map images	Locations save without a pin; maps are blank
Email verifier service	Rejecting disposable and undeliverable addresses	Registration proceeds
SMTP provider	Sending email	Email is skipped
Expo Push API	Phone notifications	In-app and email notifications still arrive
ipapi.co	Country hint for place search	Search runs without the hint

4.3 Tools and Hardware

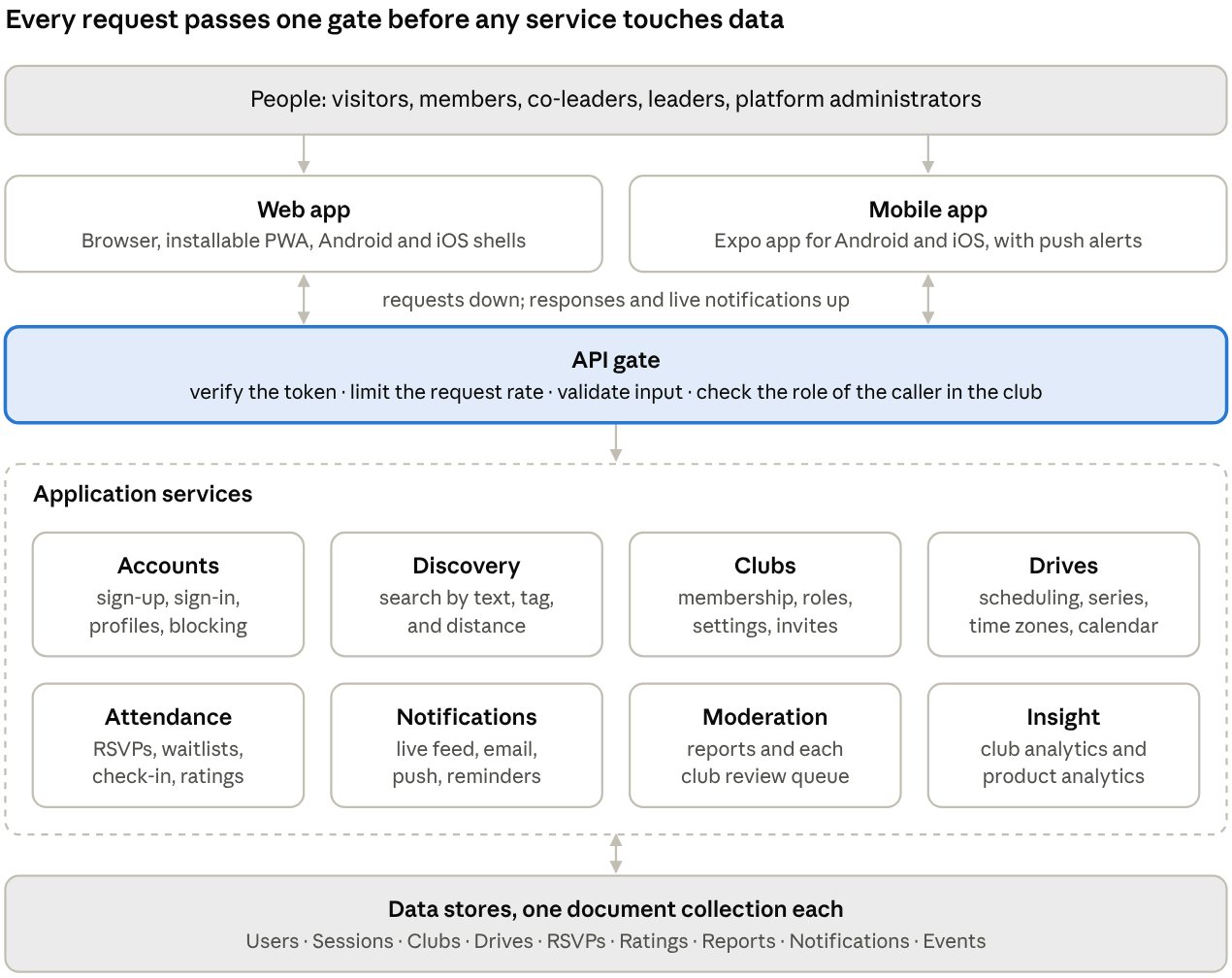
Where	What is needed
Development	A Windows 11 workstation with Node.js 20, MongoDB Community Server as a local service, Visual Studio Code, and Git with a private GitHub repository shared with the instructor. Android Studio, and a Mac with Xcode, only to build the native shells.
Testing	GitHub Actions runners for CI, and one Android phone and one iPhone for the device checks in NFR-20.
Production	Railway's managed containers and a MongoDB Atlas cluster. No hardware is owned or operated.

Where	What is needed
Users	Any computer or phone with a current browser and a screen at least 360 px wide, or an Android or iOS phone for the mobile app. Location services are optional.

5. Logical System Design

Every request from either client passes one gate (identity, rate limit, input validation, and the caller's role in the club) before it reaches one of eight application services, and every service reads and writes the same nine document collections. Section 5.1 shows that logical structure, 5.2 traces the RSVP and waitlist rules through it, 5.3 maps it onto servers and outside services, and 5.4 and 5.5 define the data it carries.

5.1 Logical Architecture

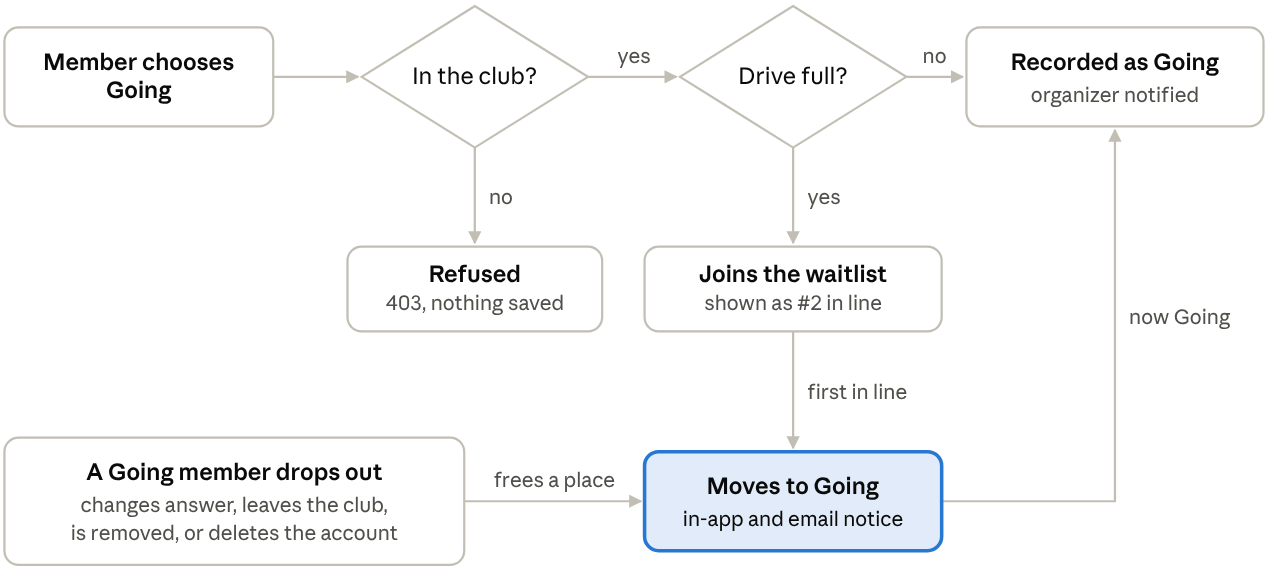


logical architecture · 5 layers, 8 services

People act through either client, and both clients speak to one API. The gate checks every request the same way, so no service repeats identity or permission logic, and outside services (geocoding, email, push, address checks) attach where Section 5.3 shows.

5.2 Information Flow: RSVP and Waitlist

A full drive puts members in line, and a freed place goes to the first in line

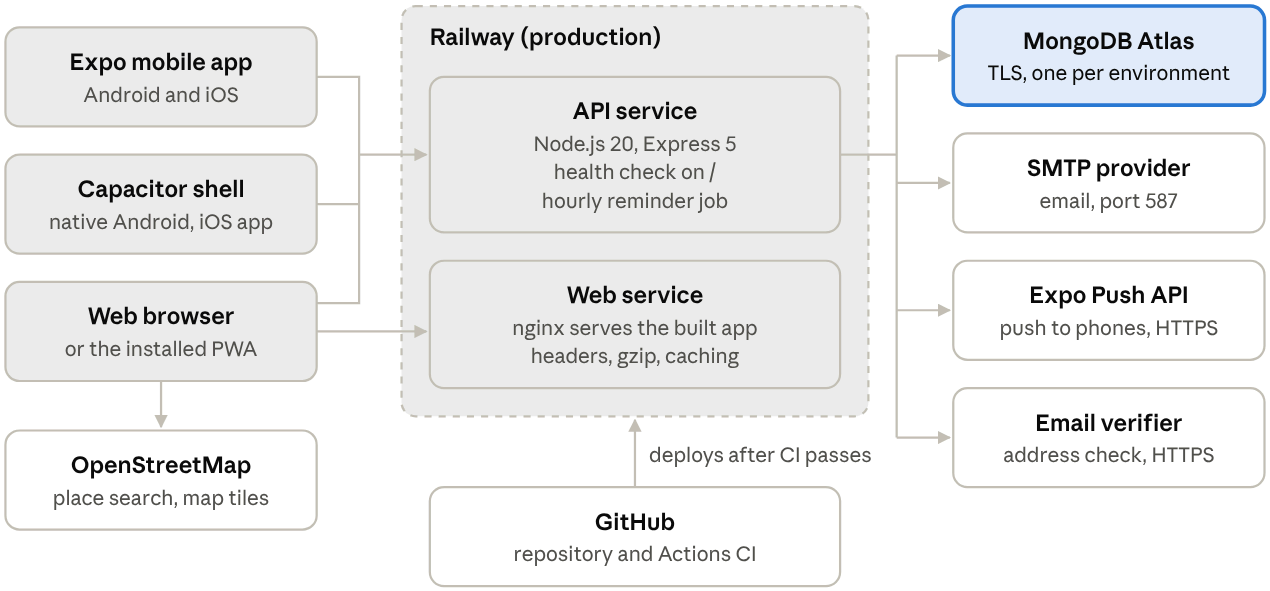


RSVP and waitlist rules · 2 decisions, 1 promotion path

The Attendance service applies these rules (FR-40, FR-41). Maybe and Not Going skip the capacity check, and the promotion path runs whatever freed the place: a changed answer, a member leaving or removed, or a deleted account.

5.3 Physical Architecture

One API on Railway serves every client; only MongoDB Atlas is a hard dependency



physical architecture · 3 clients, 2 Railway services, 5 outside services

Clients reach Railway over HTTPS on port 443: the browser loads the built app from the web service, and every client sends JSON requests and holds a Server-Sent Events stream to the API service. The API alone talks to MongoDB Atlas (TLS), the SMTP provider (port 587), the Expo Push API, and the email verifier. The browser alone calls OpenStreetMap, and ipapi.co for a country hint in place search. The Capacitor shells carry the built app inside them, so they skip the web service.

5.4 Data Entities

Entity	Key fields	Relationships and bounds
User	Username, email, password hash, profile, garage, theme, notification settings, push tokens, blocked users and clubs, recent password hashes	Belongs to clubs; owns RSVPs, ratings, reports, notifications, events, sessions. 5 cars with 4 photos, 10 push tokens
RefreshToken	Token hash, user, expiry, revoked flag	One user. Deleted at expiry (TTL, 7 days)

Entity	Key fields	Relationships and bounds
Club	Name, description, location and point, avatar, privacy, tags, member limit, invite code, leader, co-leaders, members, join requests, bans, announcements	Many drives; reviews reports filed in it. 3 co-leaders, 5 tags, unique name
Drive	Name, local day and time, time zone, start instant, place and pin, description, difficulty, attendee limit, route image, photos, cancelled and completed state, series link	One club and creator; many RSVPs and ratings. 12 photos, 12 per series
RSVP	Drive, user, answer (going, maybe, not going, waitlisted), reminder sent, check-in state	Unique per member and drive
DriveRating	Drive, user, 1 to 5 stars, comment	Unique per member and drive. Comment 200 characters
Report	Reporter, target type and id, target name, reviewing club, reason, details, status, reviewer	Unique per reporter and target. Details 500 characters
Notification	User, type, message, link data, read flag	One user. No limit yet (UC-55)
Event	User, type, page path, details	One user. No limit yet (UC-55)

Waitlist position, going counts, rating averages, distances, and analytics figures are derived when read and never stored, so nothing needs renumbering or recounting when data changes.

5.5 Notification Types

Sent to	Types and when
The club's members	NEW_DRIVE (a drive or series is scheduled), DRIVE_CANCELLED, NEW_ANNOUNCEMENT
The drive's organizer	RSVP_NEW, RSVP_UPDATED

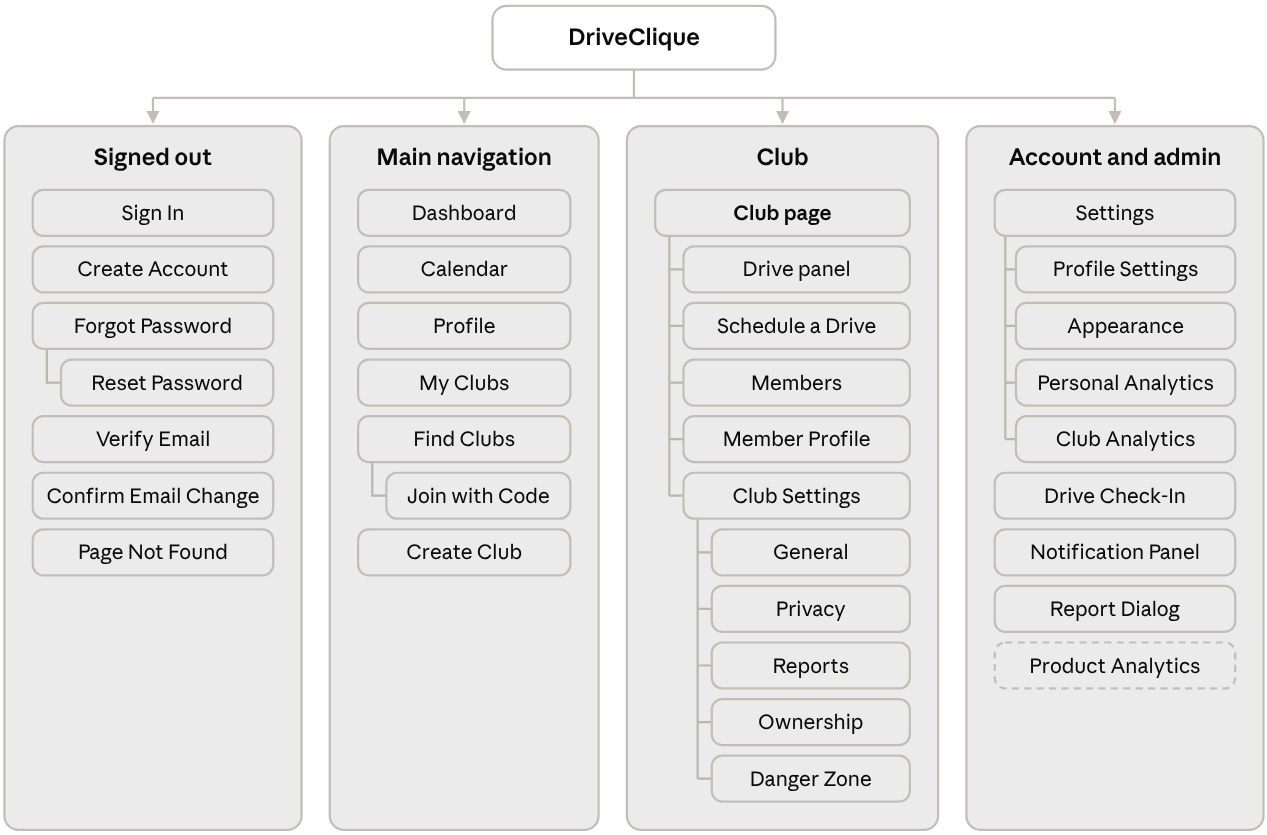
Sent to	Types and when
The member concerned	WAITLIST_JOINED, WAITLIST_PROMOTED, JOIN_ACCEPTED, JOIN_REJECTED, COLEADER_PROMOTED, COLEADER_DEMOTED
Members who answered Going or Maybe	DRIVE_REMINDER, about 24 hours before the drive
Members who answered Going	DRIVE_CHECKIN_REQUEST, DRIVE_PHOTOS_ADDED
The leader and co-leaders	JOIN_REQUEST, NEW_REPORT (never to the person reported)

6. User Interface Design

The web app has 18 routed pages and 7 overlays, reached from one persistent layout, and the mobile app has four tabs over the same API. Section 6.1 maps every screen, 6.2 describes the shared layout, 6.3 lists each screen with its route, audience, and requirements, and 6.4 gives a wireframe for each. The wireframes are low-fidelity: they fix content, order, and controls, not colour or type, and the names and figures in them are sample data.

6.1 Sitemap

Two entry points: Sign In for visitors, the left navigation for members



Indented screens open from the screen above them · dashed: platform administrators only · Sign In leads to the Dashboard

sitemap · 18 pages and 7 overlays in 4 areas

A visitor enters at Sign In or Create Account and lands on the Dashboard. From there the left navigation reaches every main page, the club page reaches everything about one club, and the avatar menu reaches Settings. The Drive Check-In page opens from its notification, and the bell panel and Report dialog open over any page.

6.2 Shared Layout

Every signed-in page shares three regions. A top bar holds the logo, a search field, the notification bell with its unread count, and the avatar menu (Profile, Settings, Sign Out). A left navigation lists Home, Calendar, and Profile, then My Clubs, Find Clubs, and the user's own clubs, with a card for the signed-in user at the foot. The main column holds the page, and pages with secondary actions add a right panel. Below 1280 px the side panels collapse into drawers opened from buttons at the screen's edge, and every dialog keeps keyboard focus inside it until closed.

6.3 Screen Catalog

Wireframe	Screen	Route	Who sees it	Requirements
W01	Sign In	`/login`	Visitor	FR-02
W02	Create Account	`/register`	Visitor	FR-01
W03	Forgot and Reset Password	`/forgot-password`, `/reset-password`	Visitor	FR-05
W04	Email Verified, Email Changed, Page Not Found	`/verify-email`, `/confirm-email-change`, any unknown path	Anyone with the link	FR-04, FR-08
W05	Dashboard	`/dashboard`	Signed-in user	FR-14, FR-16, FR-47
W06	Calendar	`/calendar`	Signed-in user	FR-40, FR-48
W07	My Clubs	`/my-clubs`	Signed-in user	FR-23
W08	Find Clubs	`/find-club`	Signed-in user	FR-16, FR-17, FR-18, FR-25
W09	Join with Invite Code	`/find-club` (screen)	Signed-in user	FR-23
W10	Create Club	`/create-club`	Signed-in user	FR-19
W11	Club page, leader view	`/club/:clubId`	Member, co-leader, leader	FR-22, FR-24, FR-28, FR-34, FR-36, FR-37
W12	Club page, non-member view	`/club/:clubId`	Signed-in non-member	FR-20, FR-21, FR-25
W13	Drive panel	Overlay on any drive list	Member and above	FR-39 to FR-45, FR-49
W14	Schedule a Drive	Overlay on the club page	Co-leader, leader	FR-34, FR-35
W15	Members list and member profile	Overlays on the club page	Member and above	FR-12, FR-15, FR-29, FR-30

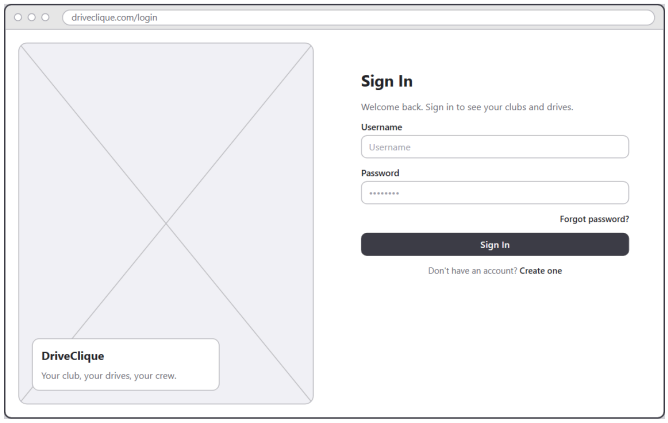
Wireframe	Screen	Route	Who sees it	Requirements
W16	Club Settings: General	`/club/:clubId/settings/general`	Leader	FR-26
W17	Club Settings: Privacy, Ownership, Danger Zone	`/club/:clubId/settings/:section`	Leader	FR-27, FR-31, FR-32
W18	Club Settings: Reports	`/club/:clubId/settings/reports`	Co-leader, leader	FR-56
W19	Profile and Garage	`/profile`	Signed-in user	FR-10, FR-11
W20	Profile Settings	`/settings`	Signed-in user	FR-06 to FR-09, FR-51
W21	Appearance and Analytics	`/settings` (views)	Signed-in user; Clubs view for leaders and co-leaders	FR-13, FR-33, FR-46, FR-49
W22	Drive Check-In	`/drive/:driveId/checkin`	Member who answered Going	FR-43
W23	Notification panel and Report dialog	Overlays on any page	Signed-in user	FR-50, FR-55
W24	Product Analytics	`/admin/analytics`	Platform administrator	FR-57
W25	Mobile app: Dashboard, Club, Check-In	Expo tabs	Mobile user	FR-40, FR-43, FR-47, FR-54

6.4 Wireframes

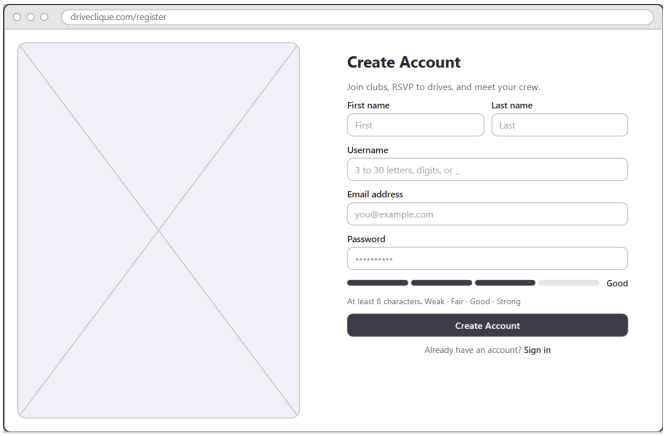
Four wireframes share each figure, each labeled with its number; zoom in on the PDF for detail.

Figure 6.1: W01 to W04, Signing In and Account Links

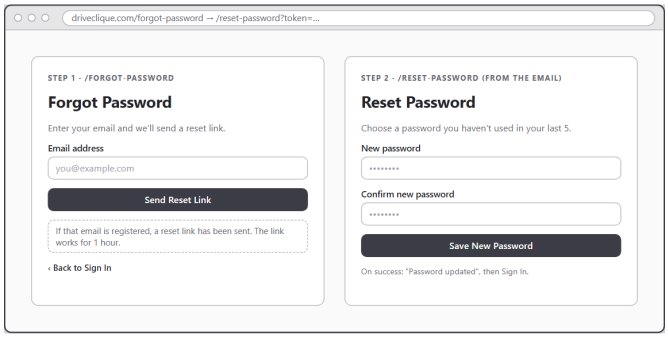
W01 Sign In



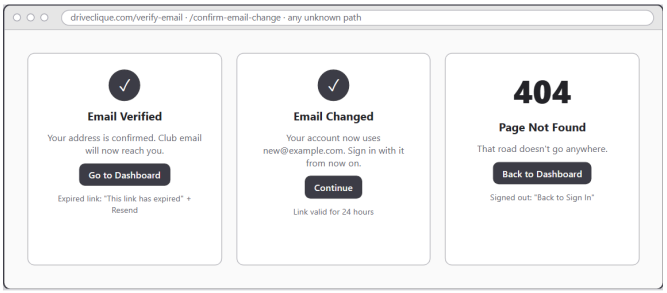
W02 Create Account



W03 Forgot and Reset Password



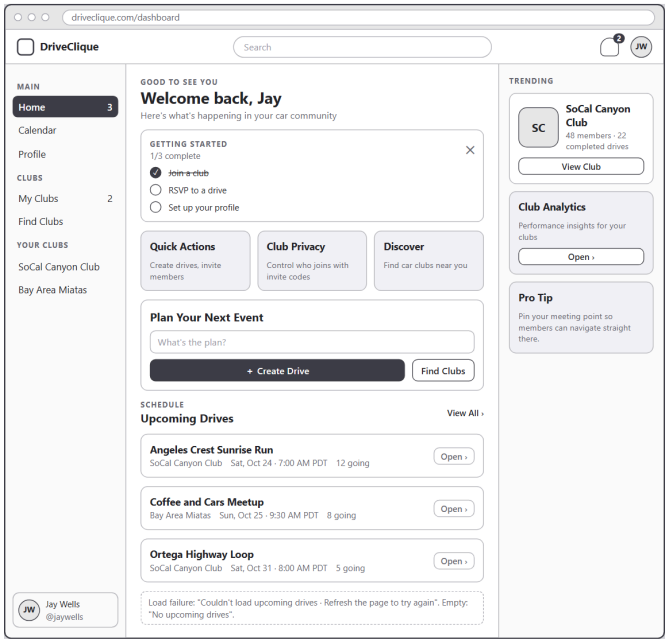
W04 Link Pages, Page Not Found



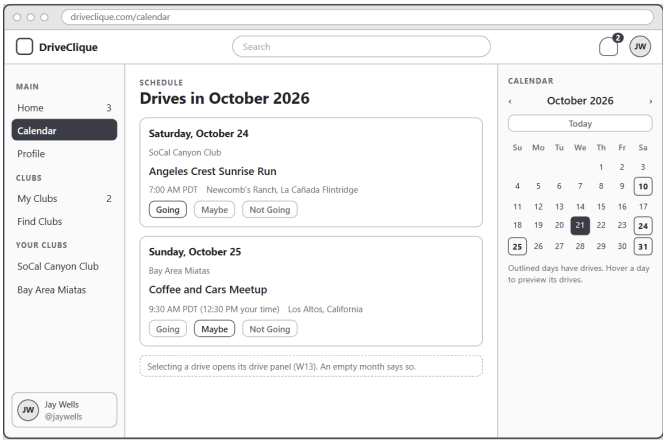
W01 Sign In. A photo panel beside a short form, with links to Forgot Password and Create Account. **W02 Create Account.** The strength meter fills in four steps as the password is typed, and errors appear under the field that caused them. **W03 Forgot and Reset Password.** The first card never says whether an address has an account; the second opens from the emailed link and refuses the last five passwords. **W04 Link Pages.** One-purpose pages reached from emailed links, and a branded 404 that sends signed-in and signed-out users to different places.

Figure 6.2: W05 to W08, Main Navigation

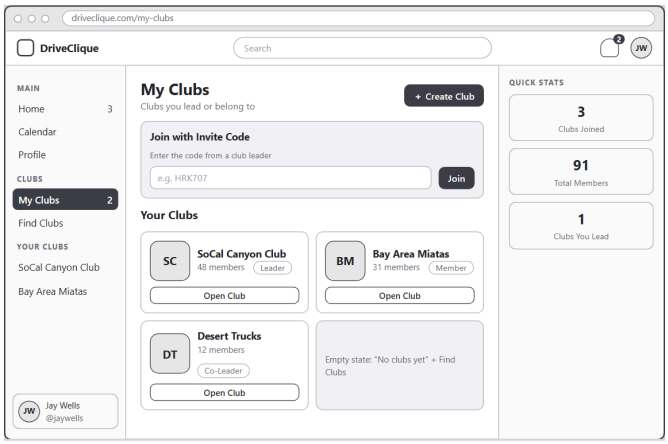
W05 Dashboard



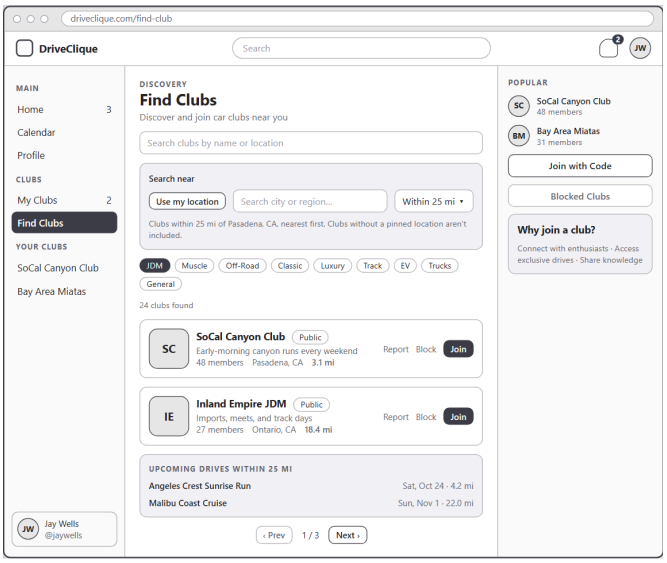
W06 Calendar



W07 My Clubs



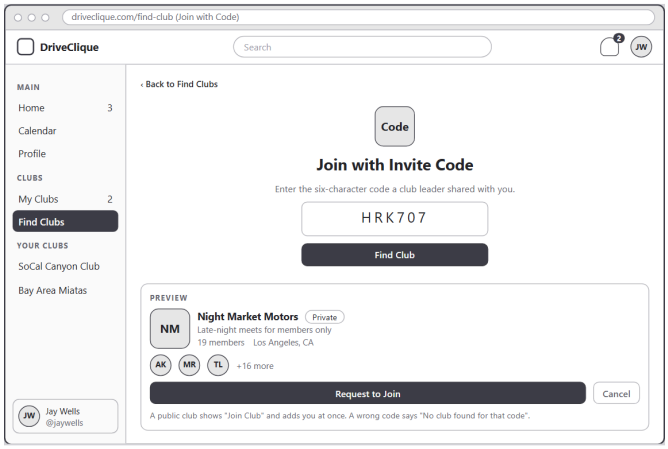
W08 Find Clubs



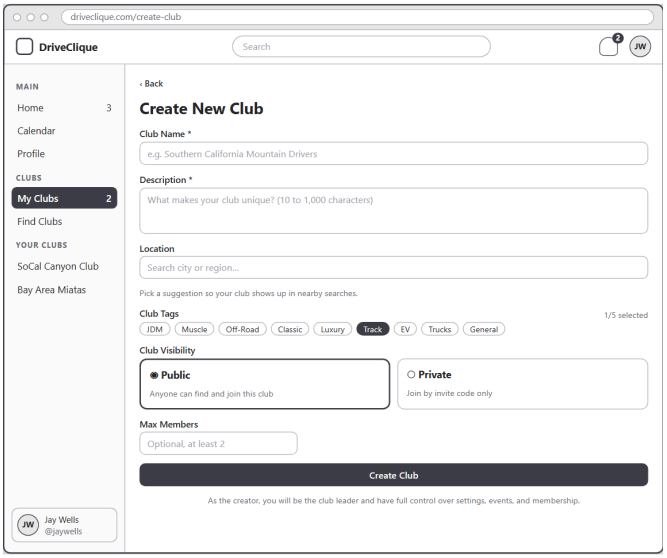
W05 Dashboard. The Getting Started checklist leads, then upcoming drives from every club from one request; the right panel holds the trending club, the analytics shortcut, and a tip. **W06 Calendar.** The month grid outlines days with drives, and the main column lists them with RSVP buttons and the viewer's own time when it differs. **W07 My Clubs.** Every club the user belongs to, with their role, plus invite-code entry and totals. **W08 Find Clubs.** Text, distance, and tag filters narrow one paged list; each result offers Report, Block, and Join, and nearby drives follow.

Figure 6.3: W09 to W12, Joining and Club Pages

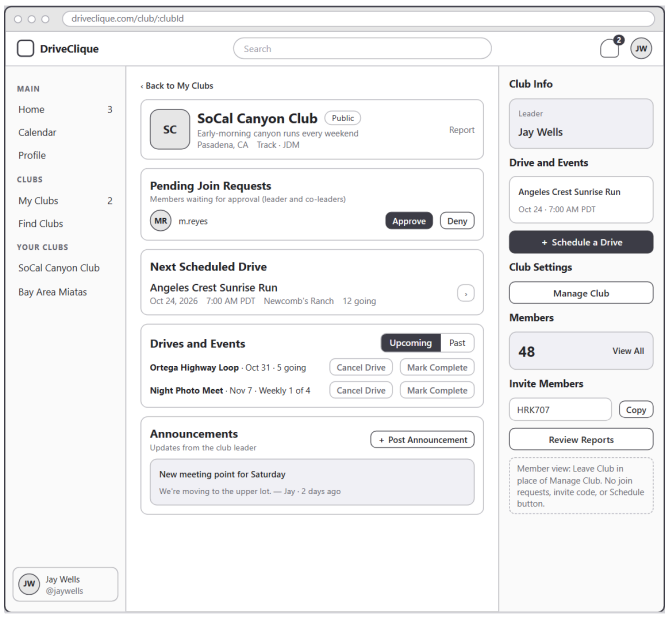
W09 Join with Invite Code



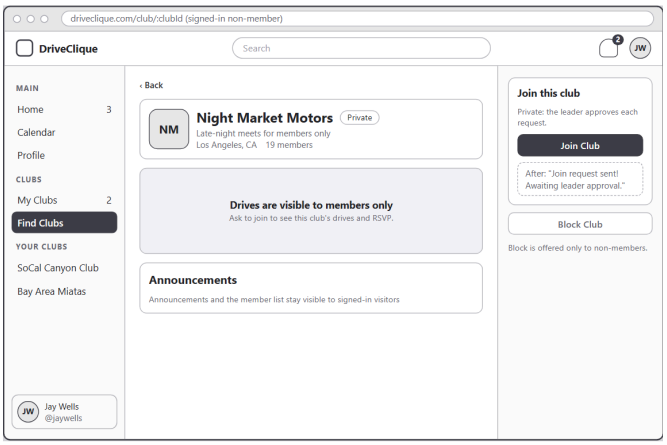
W10 Create Club



W11 Club Page, Leader View



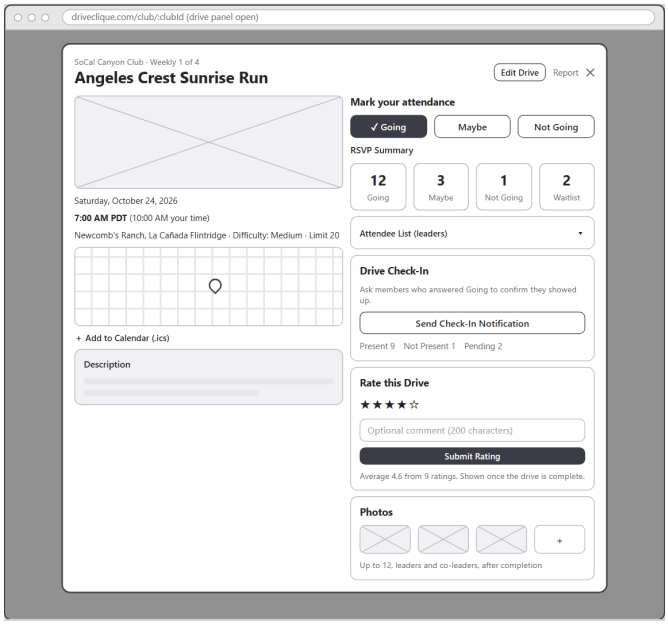
W12 Club Page, Non-Member



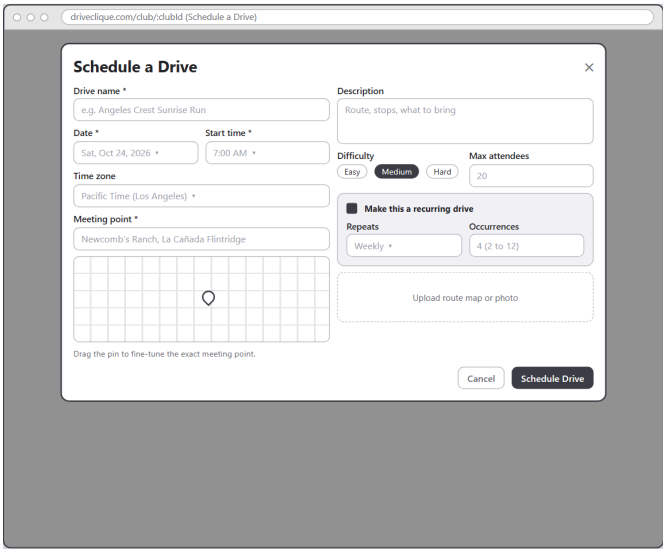
W09 Join with Invite Code. A code shows the club before anyone joins; a private club offers Request to Join. **W10 Create Club.** Required fields are marked, and the hint under Location says whether the club is pinned for distance search. **W11 Club Page, Leader View.** Join requests, the next drive, all drives, and announcements in the main column; the leader's actions on the right. A member sees the same page without the leader's controls. **W12 Club Page, Non-Member View.** A private club hides its drives and offers Join Club and Block Club.

Figure 6.4: W13 to W16, Drives, Members, and Club Settings

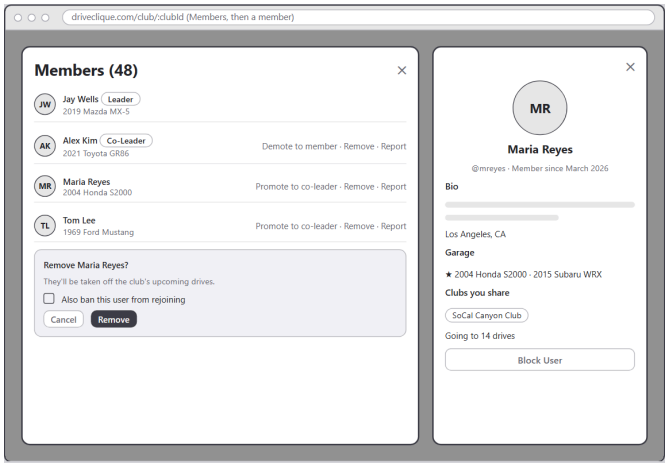
W13 Drive Panel



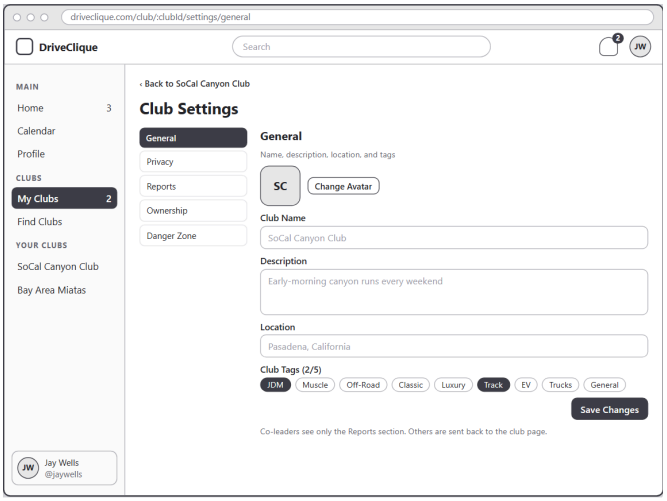
W14 Schedule a Drive



W15 Members and Profile



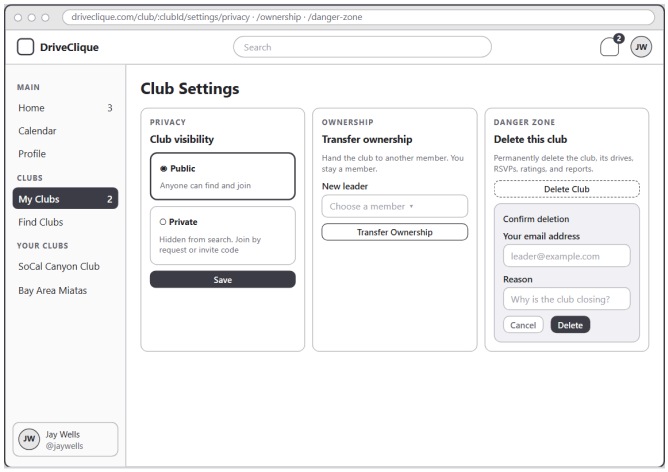
W16 Club Settings: General



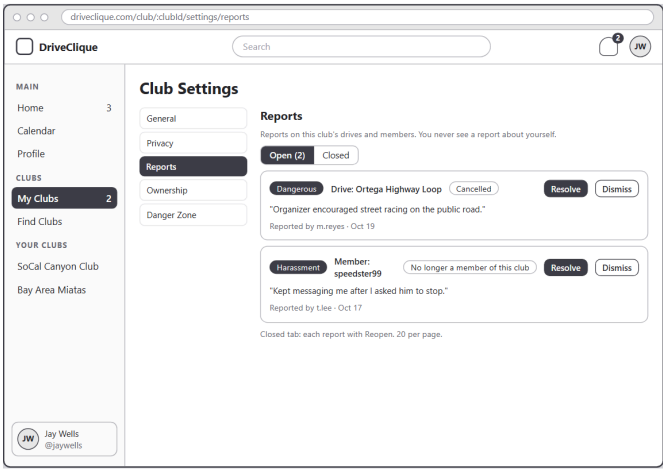
W13 Drive Panel. What and where on the left, what the viewer can do on the right; check-in appears for leaders, ratings and photos once the drive is complete. **W14 Schedule a Drive.** When and where with the draggable pin on the left; details, repetition, and the route image on the right. Edit Drive reuses this layout. **W15 Members and Profile.** Each row shows role, primary car, and the actions the viewer's role allows; a name opens the profile with Block User. **W16 Club Settings: General.** One URL per section, each saved on its own; co-leaders see only Reports.

Figure 6.5: W17 to W20, Club Settings and Profile

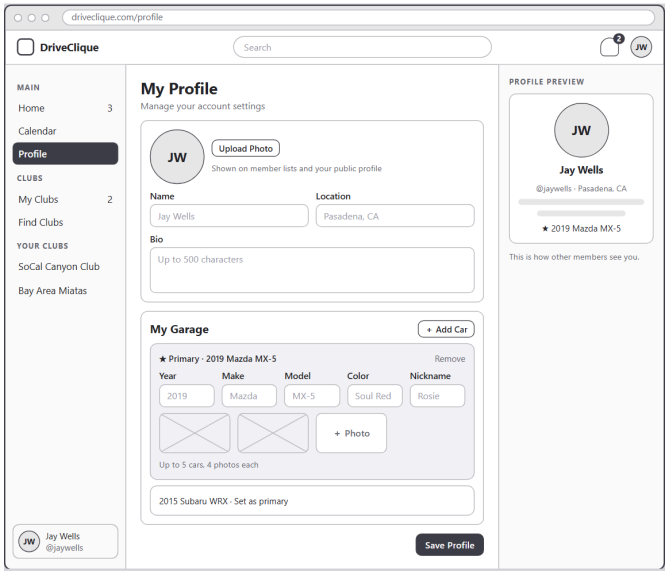
W17 Privacy, Ownership, Danger Zone



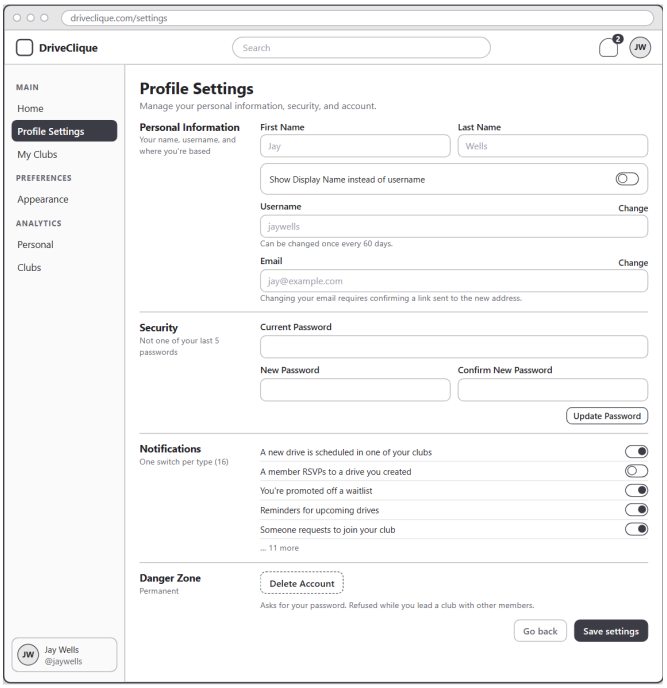
W18 Club Settings: Reports



W19 Profile and Garage



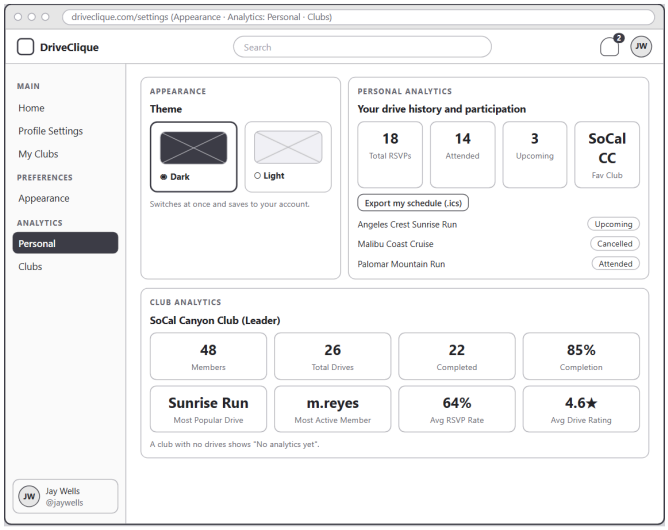
W20 Profile Settings



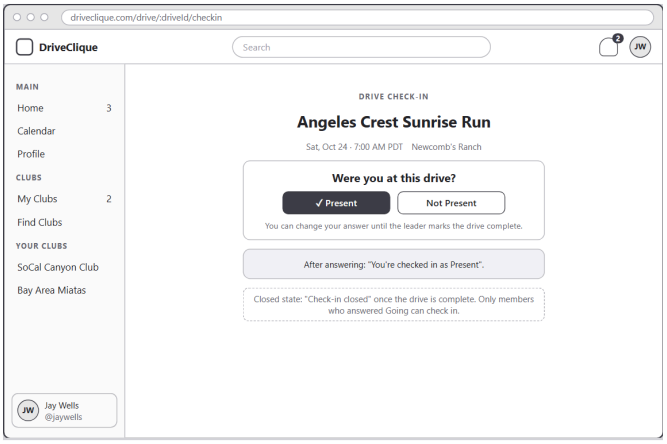
W17 Privacy, Ownership, Danger Zone. The leader-only sections; deleting a club asks for the leader's email address and a reason. **W18 Reports.** Open and Closed queues, each report showing its target's current state. **W19 Profile and Garage.** Profile and garage edit together beside a live preview. **W20 Profile Settings.** Personal information, security, notification switches, and the Danger Zone.

Figure 6.6: W21 to W24, Settings, Check-In, Overlays, and Admin

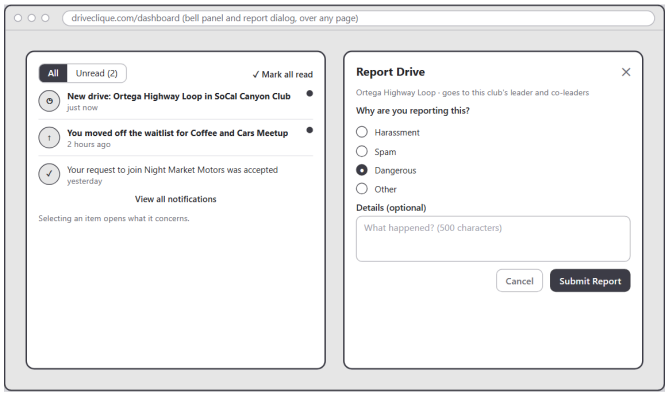
W21 Appearance and Analytics



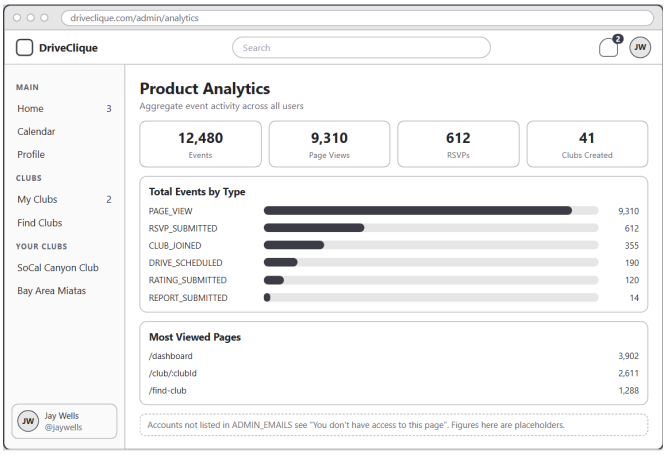
W22 Drive Check-In



W23 Notifications, Report Dialog

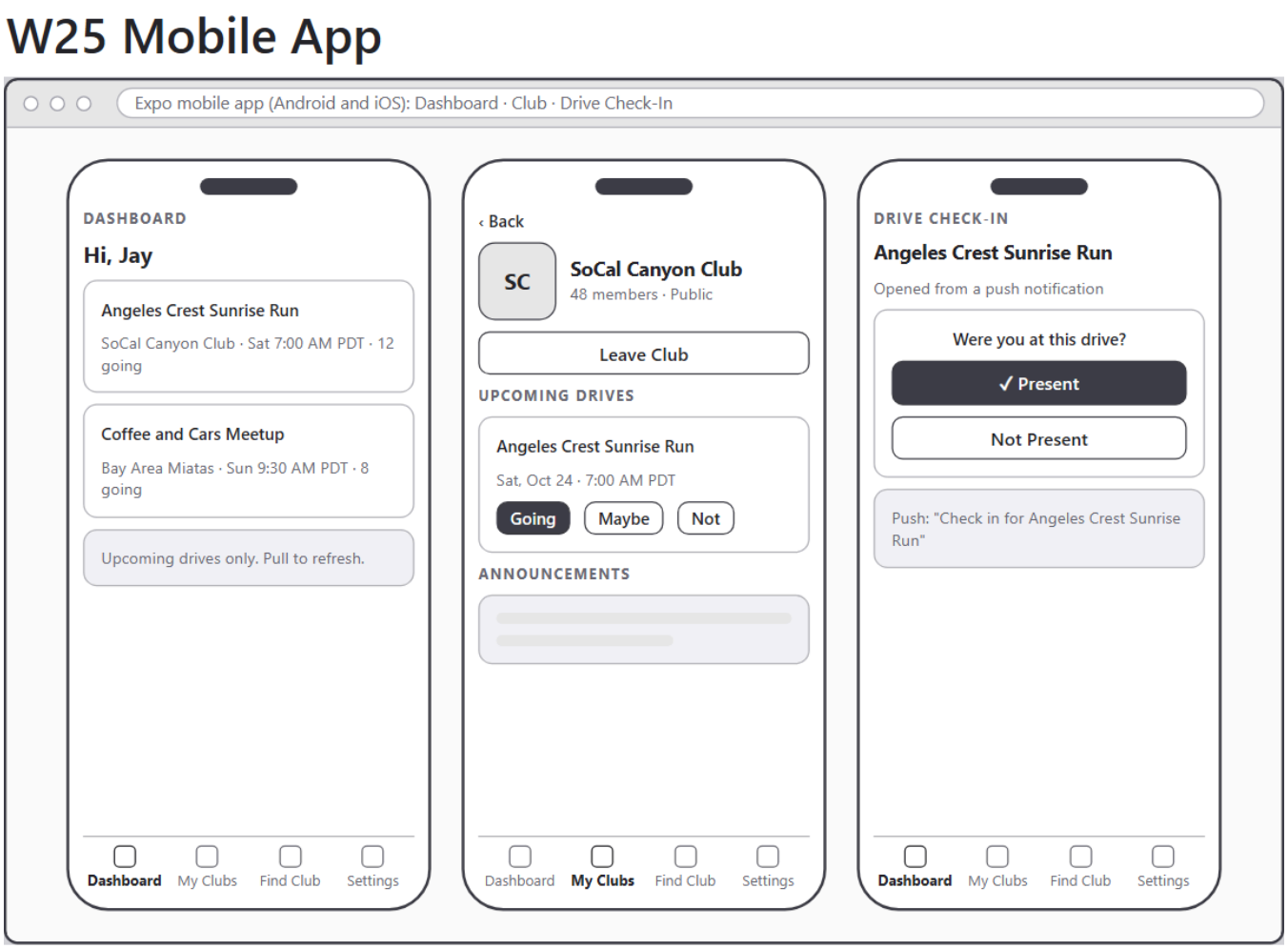


W24 Product Analytics



W21 Appearance and Analytics. The theme applies at once; personal analytics holds the RSVP history and schedule export, and club analytics appears for leaders and co-leaders. **W22 Drive Check-In.** Opened from its notification: one question, two answers, and a closed state after completion. **W23 Notifications and Report Dialog.** The bell panel filters to Unread and marks all read; the report dialog names who will review it. **W24 Product Analytics.** Totals by event type and the most viewed pages, for platform administrators only.

Figure 6.7: W25, Mobile App



W25 Mobile App. The Expo app's four tabs (Dashboard, My Clubs, Find Club, Settings) cover the core flows, and a push notification opens the matching screen, such as check-in.

Appendix A. Glossary

Term	Meaning
Club	A group of members with one leader, its own drives, and a privacy setting
Drive	A scheduled group outing with a start time, a time zone, a meeting point, and an attendee limit

Term	Meaning
Series	Up to 12 drives created together on a weekly, every-two-weeks, or monthly pattern
RSVP	A member's answer to a drive: Going, Maybe, or Not Going, or Waitlisted when full
Waitlist	The ordered list of members waiting for a place on a full drive
Leader, co-leader	The club's owner, and up to three members the owner trusts with day-to-day duties
Invite code	A six-character code that lets someone find and join a club, including a private one
Access token, refresh token	A 15-minute signed proof of identity, and the 7-day token that renews it
SSE	Server-Sent Events, a one-way stream from server to browser over HTTP
PWA	Progressive Web App: a website that can be installed and opened like an app
IANA time zone	A named zone such as America/Los_Angeles that includes its daylight-saving rules
.ics	The standard calendar file format (iCalendar) read by Google, Apple, and Outlook calendars
TTL index	A database index that deletes each document when its expiry time passes
MoSCoW	Priority levels: Must, Should, Could, and Won't for now

Appendix B. Revision History

Revision	Date	Change
1.0	October 21, 2026	First issue for Milestone 2: functional and non-functional user stories, technical requirements, logical and physical design, sitemap, and wireframes